

Cliente DNS MX em C, passo a passo

Fundamentos de Redes de Computadores • Trabalho 01 • 2026.2

Base: enunciado do Prof. Tiago Alves, arquivo trabalho_01_2026.02(2).pdf, quatro páginas.
Guia preparado em 29/09/2026.

A solução escolhida

Um programa de terminal, escrito em C, que recebe domínio e IP do servidor, monta uma consulta DNS MX byte a byte, envia por UDP à porta 53 e interpreta a resposta. Um arquivo **meu_cliente.c** e um **README.md** bastam para a entrega.

C exige mais cuidado com buffers do que Python, mas mantém 100% da pontuação prevista. Python e Java ficam limitados a 90%; C++ e Go, a 95%. Entre as opções com pontuação integral, C combina diretamente com GCC e a interface de sockets citados no trabalho. Rust também vale 100%, mas não reduz o esforço inicial deste exercício.

O que este documento entrega

Um roteiro de implementação com contratos de funções, trechos de C, algoritmos, decisões, testes e checklist de entrega. Os trechos são peças para construir o programa; **não constituem, isoladamente, um código-fonte completo pronto para compilar.**

Etapa	Páginas
Conferir requisitos e preparar o ambiente	2-3
Montar consulta e controlar comunicação UDP	4-6
Descomprimir nomes e interpretar registros MX	7-9
Testar, documentar e empacotar	10-12

Escopo mínimo

Sem interface gráfica, banco de dados, threads, cache, biblioteca DNS ou consulta HTTP. O cliente não percorre servidores raiz: pede recursão ao servidor indicado. A implementação sugerida usa Linux e aceita IP literal IPv4 ou IPv6. Os nomes são fornecidos em ASCII, incluindo Punycode já convertido quando necessário.

As exigências, sem omissões

O enunciado exige	Como implementar
Argumentos na ordem domínio e IP	<code>argc == 3</code> ; <code>argv[1]</code> é o domínio; <code>argv[2]</code> é IP literal.
Somente MX / classe IN	<code>QTYPE = 15</code> ; <code>QCLASS = 1</code> .
ID aleatório de 16 bits	Ler 2 bytes de <code>/dev/urandom</code> uma vez por consulta lógica.
Consulta recursiva	Flags <code>0x0100</code> ; uma pergunta; demais contadores zerados.
Socket UDP, porta 53	<code>SOCK_DGRAM</code> ; <code>sendto</code> e <code>recvfrom</code> ; porta de destino 53.
2 segundos; até 3 tentativas	Três envios no total; prazo monotônico por tentativa.
Interpretar payload de resposta	Validar cabeçalho, pergunta, nomes e registros.
Sucesso em stdout	Uma linha <code>dominio <> servidor_email</code> para cada MX.
Três situações de falha exemplificadas	Separar <code>NXDOMAIN</code> , ausência de MX e falha de coleta.
Fonte + documentação, em ZIP	Sem executáveis; README ou PDF com os seis itens pedidos.

Como interpretar os trechos ambíguos

“Entradas do teclado” é concretizado pelos argumentos da linha de comando nos exemplos. Não é necessário criar um menu ou usar `scanf`. O item “Caso o nome apresentado pelo usuário não exista,” está incompleto; o exemplo de domínio não encontrado esclarece o comportamento esperado.

A permissão para bibliotecas auxiliares não autoriza delegar a consulta a um resolvedor pronto. O aviso final exige montar o payload programaticamente. Não usar `res_query`, `getaddrinfo` para resolver o domínio consultado, `dnspython`, `c-ares` nem executar `dig` dentro do cliente. O guia também faz a leitura da resposta manualmente, embora o texto permita funções auxiliares para extração.

Prazo e envio: o enunciado fornecido não informa data-limite. A entrega é no ponto de coleta do Sigaa. O material acompanhante do Moodle não veio no anexo; confira-o antes de entregar.

1. Preparar o projeto

Ambiente de referência: Linux com GCC. Se desenvolver no Mac, faça a verificação final em Linux disponível para a dupla ou no laboratório. No Windows, WSL com Linux é uma opção. Registre no README o ambiente realmente utilizado, sem copiar uma versão que não testou.

```
mkdir cliente_dns
cd cliente_dns
# Crie meu_cliente.c e README.md no editor de sua escolha.
gcc -std=c11 -D_POSIX_C_SOURCE=200809L \
    -Wall -Wextra -Wpedantic -O2 meu_cliente.c -o meu_cliente
./meu_cliente unb.br 8.8.8.8
```

Se gcc não estiver instalado em Debian/Ubuntu, o pacote build-essential fornece as ferramentas de compilação. Nenhuma biblioteca externa nem flag de ligação especial é necessária na proposta Linux.

Cabeçalhos / recurso	Por que usar
stdint.h, stddef.h	Inteiros de tamanho explícito e índices size_t.
stdio.h, string.h, stdlib.h	Console, buffers, comparação e funções básicas.
sys/socket.h, netinet/in.h, arpa/inet.h	Sockets, endereços, htons e inet_pton.
poll.h, time.h, errno.h	Espera com prazo e tratamento de interrupções.
unistd.h	close para liberar o socket.
/dev/urandom via fopen/fread	ID aleatório sem instalar biblioteca; falhar se a leitura falhar.

Divisão sugerida dentro de meu_cliente.c

```
put16 / get16          // inteiros em ordem de rede
encode_qname           // nome textual -> labels DNS
make_query             // cabeçalho + pergunta
read_name              // labels e ponteiros de compressao
parse_response         // valida e classifica resposta
now_ms / exchange     // prazo, envios e recepcao
main                   // argumentos, resultado e fechamento
```

Faça main pequeno. Funções de protocolo recebem sempre o buffer e seu tamanho. Não esconda limites em variáveis globais. Use enum para distinguir sucesso, ausência de MX, NXDOMAIN, resposta irrelevante e erro.

2. Montar o pacote DNS

Use um vetor `uint8_t query[512]` inicialmente zerado. Escreva os campos explicitamente: transmitir uma struct pode introduzir preenchimento e ordem de bytes incorretos. Este é o payload DNS; o sistema operacional monta os cabeçalhos IP e UDP.

Offset	Campo	Valor da consulta
0	ID	16 bits aleatórios
2	FLAGS	0x0100
4	QDCOUNT	0x0001
6 / 8 / 10	AN / NS / ARCOUNT	0x0000 em cada campo
12	QNAME	Domínio em labels, terminado por zero
Após QNAME	QTYPE / QCLASS	0x000F / 0x0001

```
static void put16(uint8_t *p, uint16_t v) {
    p[0] = (uint8_t)(v >> 8);
    p[1] = (uint8_t)v;
}
static uint16_t get16(const uint8_t *p) {
    return (uint16_t)(((uint16_t)p[0] << 8) | p[1]);
}

uint16_t id;
FILE *rng = fopen("/dev/urandom", "rb");
if (!rng) { /* informar erro local e encerrar */ }
/* So continuar se fread retornar exatamente 2. */
size_t got = fread(&id, 1, sizeof id, rng);
fclose(rng);
if (got != sizeof id) { /* informar erro e encerrar */ }

memset(query, 0, sizeof query);
put16(query, id);
put16(query + 2, 0x0100);
put16(query + 4, 1);
/* encode_qname escreve em query + 12 e devolve qlen. */
put16(query + 12 + qlen, 15);
put16(query + 14 + qlen, 1);
size_t query_len = 16 + qlen;
```

Pré-condições: incluir os cabeçalhos; declarar `query` e `qlen`; validar o retorno de `encode_qname` antes de escrever `QTYPE` e `QCLASS`. `get16` só deve ser chamado depois de conferir que há pelo menos dois bytes disponíveis.

Para unb.br, `QNAME` ocupa 8 bytes e a consulta ocupa 24. Para inspecionar manualmente, fixe o ID apenas no teste como 0x1234:

```
12 34 01 00 00 01 00 00 00 00 00 00
03 75 6e 62 02 62 72 00 00 0f 00 01
```

3. Codificar o nome consultado

DNS não transmite “unb.br” como uma string com pontos. Transmite o comprimento e os bytes de cada parte: 3, u, n, b, 2, b, r, 0. O zero final encerra o nome. Não envie o caractere de ponto como separador.

Contrato de encode_qname

```
int encode_qname(const char *name, uint8_t *dst,
                 size_t cap, size_t *written);
/* Retorno: 0 em sucesso, -1 em entrada/capacidade invalida. */
```

- 1. Copie a entrada para um buffer limitado. Remova no máximo um ponto final, permitindo unb.br. como entrada equivalente a unb.br. Rejeite nome vazio e rótulos vazios como em a..br.
- 2. Percorra os componentes sem usar strtok: ele pode esconder campos vazios. Para cada componente, calcule o comprimento, confira 1 a 63 bytes e só então escreva comprimento e conteúdo.
- 3. Valide que o nome completo codificado, incluindo comprimentos e zero terminal, cabe em 255 bytes. Confira também a capacidade do destino antes de cada escrita.
- 4. Use ASCII para o escopo do trabalho. Rejeite espaços, controles e bytes não ASCII; documente que nomes internacionalizados precisam vir como Punycode. Para nomes de domínio usuais, aceite letras, números e hífen, sem hífen nas pontas de cada componente.
- 5. Acrescente 0x00 e retorne a quantidade de bytes efetivamente escrita. A saída textual pode usar a versão normalizada, sem ponto final.

Verificação pequena antes de implementar a rede

Entrada	Resultado esperado
unb.br	03 75 6e 62 02 62 72 00
unb.br.	Mesmo QNAME de unb.br
a..br ou string vazia	Erro de argumento; não enviar pacote
Rótulo de 64 bytes	Rejeitar antes de escrever no buffer
Nome codificado acima de 255 bytes	Rejeitar com mensagem clara

Valide o pacote antes de testar a rede: um nome mal codificado pode ser descartado pelo servidor e parecer um timeout.

Checkpoint: produza os 24 bytes do exemplo sem rede. Depois envie query_len bytes, nunca sizeof(query).

4. Enviar e esperar por UDP

Construa o endereço com `inet_pton`: tente `AF_INET` e depois `AF_INET6`; ambos devem receber porta `htons(53)`. Use `sockaddr_storage` para armazenar qualquer um e mantenha o tamanho correto da estrutura. Se nenhuma conversão retornar 1, o IP informado é inválido. `inet_pton` converte texto; não consulta DNS. [4]

```
int fd = socket(family, SOCK_DGRAM, 0);
/* Testar fd < 0 antes de prosseguir. */
ssize_t sent = sendto(fd, query, query_len, 0,
                      (struct sockaddr *)&server, server_len);
/* Sucesso somente se sent == (ssize_t)query_len. */
```

Não faça `bind` na porta local 53 e não execute o cliente como administrador. O sistema escolhe uma porta de origem temporária. Use `recvfrom` para obter também o endereço de quem respondeu. Antes de cada chamada, reinicialize o tamanho da estrutura de origem. [5]

Algoritmo de exchange: três envios, não quatro

```
para tentativa = 1 ate 3:
    enviar o mesmo pacote para o mesmo servidor
    prazo = agora_monotonico_em_ms() + 2000
    enquanto agora < prazo:
        restante = prazo - agora
        poll(fd, POLLIN, restante)
        se interrompido por EINTR: recalculer restante
        se prazo esgotado: sair deste enquanto
        se erro local irrecuperavel: encerrar com erro
        receber um datagrama com recvfrom
        conferir IP e porta de origem
        conferir ID e pergunta; descartar os alheios
        se resposta valida da consulta: devolver resultado
    // Sem resposta valida: proxima tentativa
retornar FALHA_COLETA
```

Use `clock_gettime(CLOCK_MONOTONIC, ...)` para `now_ms` e `poll` para aguardar. O timeout de `poll` é em milissegundos. Recalculer o restante impede que um pacote alheio reinicie a espera de dois segundos. [3]

Compare origem campo a campo (família, endereço e porta; escopo quando aplicável ao IPv6), não a estrutura inteira com `memcmp`. Preserve socket e ID entre retransmissões: uma resposta atrasada ainda é da mesma consulta.

Sem resposta, o total será aproximadamente seis segundos; escalonamento do sistema pode acrescentar pequena variação. `NXDOMAIN` e ausência de `MX` são respostas válidas: não é preciso repetir. Erros locais devem gerar diagnóstico em `stderr`; nenhum caminho pode esperar indefinidamente. Sempre feche `fd`.

5. Ler nomes comprimidos

A resposta pode substituir um sufixo por um ponteiro de dois bytes. Se os dois bits superiores são 11, os 14 bits restantes são um offset a partir do início da mensagem DNS. Exemplo: C0 0C aponta para o byte 12. O nome pode misturar labels e um ponteiro. [1, seção 4.1.4]

Separe posição de leitura e posição de continuação

```
int read_name(const uint8_t *msg, size_t n, size_t *off,
              char *out, size_t cap);
/* off entra no início do nome e sai apos sua representacao
   original; out recebe o nome expandido terminado em \0. */
```

O erro mais comum é mover o cursor do registro para dentro do sufixo referenciado. O ponteiro muda a posição usada para ler o nome, mas o próximo campo continua logo depois dos dois bytes do ponteiro original.

```
pos = *off; next = indefinido; saida = vazia
repetir com limite de n passos:
    exigir pos < n; b = msg[pos]
    se b == 0:
        se next indefinido: next = pos + 1
        terminar string; *off = next; retornar sucesso
    se (b & 0xC0) == 0xC0:
        exigir pos + 1 < n
        destino = ((b & 0x3F) << 8) | msg[pos + 1]
        exigir destino < n
        se next indefinido: next = pos + 2
        pos = destino
    senao:
        rejeitar (b & 0xC0) != 0
        exigir 1 <= b <= 63 e pos + 1 + b <= n
        acrescentar ponto se houver label anterior
        copiar b bytes, preservando espaco para \0
        pos = pos + 1 + b
se nao terminou dentro do limite: erro de formato
```

Além do limite de passos, limite o nome expandido ao tamanho DNS permitido. Um buffer de 256 bytes basta para a forma textual simples; não copie bytes de controle diretamente ao terminal. Para a raiz, represente a saída como “.”.

Teste que pega o erro de cursor

Coloque unb.br no offset 12 e, em outro local, 04 mail C0 0C. O resultado deve ser mail.unb.br; o cursor avança sete bytes no local original, e não até o fim do nome em 12. Teste também ponteiro fora do pacote e ponteiro para si próprio: ambos devem falhar, sem travar.

6. Validar e percorrer a resposta

Use buffer de recepção de 65535 bytes para acomodar o datagrama; só leia até o tamanho retornado por `recvfrom`. Um buffer grande não resolve truncamento DNS indicado por TC. Não assuma que todo pacote recebido é válido.

Validação antes de extrair respostas

Exija pelo menos 12 bytes. Confira ID, QR = 1, opcode = 0, QDCOUNT = 1 e a mesma pergunta: nome sem diferenciar maiúsculas, tipo MX e classe IN. Decodifique o nome com `read_name` e só leia os quatro bytes seguintes se couberem. Pacotes de outra origem ou outra consulta são ignorados dentro do prazo original.

```
uint16_t flags = get16(msg + 2);
int qr      = (flags & 0x8000) != 0;
int tc      = (flags & 0x0200) != 0;
int rcode   = flags & 0x000F;
/* Conferir limites e identidade antes de classificar. */
```

TC = 1 significa resposta truncada: não afirme que faltam MX nem imprima resultado parcial. O trabalho exige UDP, portanto documente a ausência de fallback TCP e informe falha de coleta. RCODE = 3 indica domínio inexistente; outros erros, como SERVFAIL e REFUSED, são falha de coleta, não ausência de MX.

Percorrer os registros com um cursor

```
para cada RR em Answer (ANCOUNT):
    owner = read_name(msg, n, &cursor)
    exigir cursor + 10 <= n
    type  = get16(msg + cursor)
    class = get16(msg + cursor + 2)
    // TTL ocupa cursor+4 ate cursor+7
    rdlen = get16(msg + cursor + 8)
    inicio = cursor + 10; fim = inicio + rdlen
    exigir fim <= n
    se type == 15 e class == 1:
        exigir rdlen >= 3
        preference = get16(msg + inicio)
        temporario = inicio + 2
        exchange = read_name(msg, n, &temporario)
        exigir temporario == fim
        guardar owner, preference e exchange
    cursor = fim
```

RDATA de MX contém uma preferência de 16 bits e o nome do servidor. A preferência menor tem prioridade; o formato pedido não manda imprimi-la. [1, seção 3.3.9] Use a mesma estrutura de leitura para atravessar Authority e Additional; registre SOA e NS da seção Authority para classificar respostas sem MX.

7. Classificar e imprimir

Não use `ANCOUNT == 0` como sinônimo automático de “não possui MX”. Uma resposta pode conter `CNAME`, e uma referência a outro servidor não comprova ausência de registro. Guarde os resultados antes de imprimir para não deixar saída parcial caso encontre um erro depois.

Situação validada	Ação
<code>RCODE = 3 (NXDOMAIN)</code>	Dominio X nao encontrado
<code>RCODE = 0</code> e MX do nome consultado ou de seu alias	<code>X <> servidor_email</code> , uma linha por MX
<code>RCODE = 0</code> ; nenhum MX; resposta negativa conclusiva	Dominio X nao possui entrada MX
Timeout após 3 envios; TC; erro DNS ou resposta inconclusiva	Nao foi possível coletar entrada MX para X

CNAME e resposta vazia: regra prática

Ao percorrer `Answer`, guarde também os pares `CNAME` (owner, destino) usando `read_name` no `RDATA`. Comece no domínio consultado e siga somente esses pares, com limite de oito saltos e detecção de repetição. Aceite MX cujo owner seja o nome final. Isso evita imprimir um MX sem relação com a consulta.

Se o servidor recursivo retornar a cadeia e os MX, a resolução termina no mesmo pacote. Se houver apenas aliases sem resposta final nem prova negativa, informe falha de coleta; não invente ausência de MX. A proposta não abre consultas adicionais para completar uma cadeia incompleta.

Como regra conservadora, classifique “sem MX” quando `RCODE = 0`, nenhum MX relevante e há `SOA` na seção `Authority`, sem `referral`. Respostas vazias sem evidência suficiente ficam como falha de coleta. Essa escolha pode rejeitar respostas pouco informativas, mas não confunde ausência com delegação.

Múltiplos MX e Null MX

Imprima todos os MX relevantes, na ordem recebida; ordenar por preferência é opcional. Não acrescente prioridade, `TTL` ou texto explicativo às linhas de sucesso. Um `Null MX` (preferência 0 e servidor “.”) é um registro MX que declara não receber e-mail: represente como `X <> .;` não o chame de ausência de MX. [6]

Coloque as mensagens exigidas e resultados em `stdout`. Use `stderr` para detalhes técnicos, como “resposta truncada” ou “IP inválido”. Sugestão de retorno: 0 para MX obtido e 1 para falha/ausência; documente essa convenção, que não foi imposta pelo enunciado.

8. Testar sem depender da sorte

Primeiro teste funções com bytes conhecidos. Depois teste o programa inteiro. Os domínios do enunciado são exemplos, não fixtures permanentes: os registros reais podem mudar. Um resultado diferente do exemplo não prova erro no código.

Caso	Evidência de aprovação
Consulta de unb.br com ID fixo em teste	Exatamente os 24 bytes da página 4.
MX com nome comprimido	Nome completo correto e cursor correto.
Dois ou mais MX	Uma linha por registro relevante.
NXDOMAIN sintético	Mensagem de domínio não encontrado.
NOERROR sem MX + SOA	Mensagem de ausência de MX.
CNAME + MX do destino	Resultado ligado ao domínio original.
Resposta TC / SERVFAIL / REFUSED	Falha de coleta; não ausência de MX.
Ponteiro circular, RDLENGTH inválido	Sem travar e sem leitura fora do buffer.
ID ou origem diferentes	Ignorar sem reiniciar o prazo de 2 s.
Servidor silencioso	Três consultas; espera total perto de 6 s.

Teste de integração e comparação externa

```
./meu_cliente unb.br 8.8.8.8
./meu_cliente imagdaskdasdasj.br 1.1.1.1
./meu_cliente fga.unb.br 8.8.8.8
# Opcional, somente como ferramenta de verificacao:
dig @8.8.8.8 unb.br MX +noall +answer
```

dig não pode ser chamado pelo cliente. Compare o conjunto de servidores MX, não a ordem das linhas ou um nome de servidor fixado no exemplo. Se UDP 53 externo estiver bloqueado, use um resolvidor acessível da rede e registre a condição.

Timeout controlado

Use um servidor UDP de teste local que escute a porta 53 e descarte mensagens, em ambiente isolado. Ele pode exigir privilégio para abrir essa porta; o cliente não exige. Conte os datagramas recebidos. Não dependa de 1.2.3.4 permanecer silencioso: uma rede pode devolver erro imediatamente.

Status deste guia: não houve execução do cliente completo nem consulta UDP externa. Os testes acima são critérios para a implementação da dupla, não resultados já obtidos.

9. Documentar a aplicação

A documentação vale 20% e deve descrever o programa realmente entregue. Este guia orienta a construção, mas não substitui o registro do ambiente, dos testes e das limitações da implementação final.

Estrutura mínima de README.md

```
# Cliente DNS MX - Trabalho 01

## Autores
Nome completo e matricula de cada integrante.

## Sistema operacional
Distribuicao/versao efetivamente usada.

## Ambiente de desenvolvimento
Editor, versao do GCC e ferramentas efetivamente usadas.

## Como construir
gcc -std=c11 -D_POSIX_C_SOURCE=200809L -Wall -Wextra \
    -Wpedantic -O2 meu_cliente.c -o meu_cliente

## Como executar
./meu_cliente DOMINIO IP_DO_SERVIDOR

## Instrucoes de uso e telas
Inserir transcricoes reais do terminal para sucesso,
NXDOMAIN, ausencia de MX e falha de coleta.

## Limitacoes conhecidas
Listar os limites realmente existentes no programa.

## Testes realizados
Informar casos, ambiente e resultados observados.
```

Limitações coerentes com esta proposta

Somente MX/IN; transporte UDP à porta 53; sem fallback TCP para TC = 1; sem EDNS, validação DNSSEC ou cache; nomes ASCII/Punycode sem conversão automática; servidor indicado por IP literal; sem percorrer delegações; até oito aliases presentes na resposta; cadeia incompleta é inconclusiva. A leitura de aleatoriedade usa /dev/urandom no ambiente Linux.

Restrições de rede podem impedir consultas externas. Não prometa que a execução no Windows nativo funciona com a mesma interface POSIX. Se implementar apenas IPv4, declare essa redução; a proposta descrita aceita IPv4 e IPv6 globais.

O que mostrar nas “telas”

Como a aplicação é de console, transcrições ou capturas do terminal cumprem a função de instruções de uso. Mostre o comando, a saída real e uma frase explicando o resultado. Não apresente os exemplos deste PDF como evidência de teste executado.

10. Revisar e entregar

Checklist funcional: argumentos na ordem correta; payload manual; ID aleatório; flags 0x0100; uma pergunta MX/IN; contadores de resposta zerados na consulta; UDP 53; três tentativas de dois segundos; leitura de nomes comprimidos; mensagens corretas em stdout; recursos fechados em todos os caminhos.

Checklist de qualidade: compilar sem avisos relevantes; revisar limites antes de toda leitura e escrita; testar respostas sintéticas e um servidor real acessível; retirar logs de depuração do stdout; preencher nomes, matrículas, ambiente e limitações reais.

```
# Dentro da pasta do projeto; inclua apenas os arquivos fonte
# e a documentacao. Substitua nomes e matriculas no ZIP.
zip nome_sobrenome_matricula_nome_sobrenome_matricula_trab01.zip \
    meu_cliente.c README.md
```

Não inclua meu_cliente, arquivos .o ou executáveis de teste. Se dividir o código em mais arquivos .c/.h, inclua todos e atualize o comando de compilação no README. Extraia o ZIP em uma pasta limpa e compile dali antes do envio ao Sigaa.

Fontes e pontos de consulta

Enunciado • Trabalho 01, Prof. Tiago Alves, trabalho_01_2026.02(2).pdf. Requisitos técnicos nas páginas 2-3; entrega, documentação e deflatores na página 4.

[1] RFC 1035 • Formato DNS e MX; seções 3.3.9, 4.1.1-4.1.4. <https://www.rfc-editor.org/rfc/rfc1035>

[2] RFC 1034 • Conceitos de resolução e papel do servidor recursivo. <https://www.rfc-editor.org/rfc/rfc1034>

[3] poll(2) • Espera por eventos e timeout. <https://man7.org/linux/man-pages/man2/poll.2.html>

[4] inet_pton(3) • Conversão de endereços IP literais.
https://man7.org/linux/man-pages/man3/inet_pton.3.html

[5] recv(2) • Recepção de datagramas e endereço de origem.
<https://man7.org/linux/man-pages/man2/recvfrom.2.html>

[6] RFC 7505 • Significado de Null MX. <https://www.rfc-editor.org/rfc/rfc7505.html>

Documentação técnica consultada em 29/09/2026. As escolhas de estrutura, divisão de funções e sequência de implementação são propostas deste guia. Não são exigências adicionais atribuídas ao professor.